

交互与构造

qiuzx

June 2026

近年的 NOI 赛事中出现了越来越多的交互与构造问题，但此类问题相比于传统题比较特殊，常常在训练中容易被忽视，也没有足够多的训练样本。

传统题通常需要选手在给定条件下解决一个问题，但构造/交互题则需要选手在不完备条件下，在限制下自己找到需要的条件，然后解决问题。

很多时候这种问题被认为需要灵光一现，但通过训练，仍然可以找到规律，并总结出一些技巧。

核心：规避不好的限制，得到好的或方便处理的限制

- 递归子问题
- 找规律
- 通过 checker 或判定方式找到构造
- 从理论最值入手
- 尝试人为添加性质，构造具有特定性质的解
- 对于可能无解的问题：先观察有解条件
- 逐步构造（先舍弃部分限制）
- 调整法（包括随机、直接调整）

给定 n ，构造一组 n 阶格雷码，使得相邻两个数之间不同的位在这 2^n 个差中出现次数的极差最小。

数据范围： $n \leq 25$ 。

因为具有明显的子问题性质，考虑思路：从较小规模的问题推到较大规模的问题。

因为具有明显的子问题性质，考虑思路：从较小规模的问题推到较大规模的问题。

普通的格雷码从 $n - 1$ 递推到 n ，但 n 会出现恰好两次，因此出现次数之间的极差是很大的。此时考虑计算出理论最优解，并尝试按照理论最优解中的出现次数插入 n ，考察是否可行。

因为具有明显的子问题性质，考虑思路：从较小规模的问题推到较大规模的问题。

普通的格雷码从 $n-1$ 递推到 n ，但 n 会出现恰好两次，因此出现次数之间的极差是很大的。此时考虑计算出理论最优解，并尝试按照理论最优解中的出现次数插入 n ，考察是否可行。

当 $n = 2^k$ 时，理论最优解是 0，此时每个数都出现 2^{n-k} 次。否则由于每个数的出现次数一定是偶数，而 $n \nmid 2^n$ ，所以理论最优解是 2。下面猜测理论最优解能娶到，假定 n 充分大，尝试根据理论最优解的值进行构造。

考虑从 $n-2$ 推到 n 。这样的好处就是可以回头，因为能先插一个 n 再插一个 $n-1$ ，两次反向之后回到原处，且这个过程中上面的 2 位并不矛盾。

具体来说，可以将 $n-2$ 时的答案 $s_1, s_2, \dots, s_{2^{n-2}}$ 分成偶数段 $0 = k_0 < k_1 < k_2 < \dots < k_m = 2^{n-2}$ ，其中 $(k_i, k_{i+1}]$ 之间的部分看作一段，而 $2 \mid m$ 。考虑下面的这种构造：首先是一段 $00s_1, 00s_2, \dots, 00s_{k_1}$ ，然后换到 $01s_{k_1}$ ，接上 $01s_{k_1-1}, \dots, 01s_1$ ，再换到 $11s_1$ ，接上 $11s_2, \dots, 11s_{k_1}$ 。

上面这是一轮，下一轮从 $11s_{k_1+1}$ 开始，走到 s_{k_2} 一个来回，上面分别使用 11 过去，01 回来，00 再过去，然后再从 00 开始下一轮。以此类推直到 $00s_{k_m}$ ，此时换到 $10s_{k_m}$ 然后直接一路回到 $10s_1$ 并接上开头的 $00s_1$ 。

上述构造还有一些细节上的问题，可以在开始和结尾的时候进行一些微调。在这里不过多赘述，可以参考原题题解。

直接构造的复杂度为 $2^n + 2^{n-2} + \dots = O(2^n)$ ，可以通过。

给定一棵 n 个点的树的欧拉序的一部分，在剩余的部分中填数，使得得到的序列是某棵树的一个合法欧拉序。

数据范围： $n \leq 5 \times 10^5$ 。

从判定问题入手。先考虑如何判定一个序列是否是一个合法的欧拉序。当然可以模拟 dfs 的过程直接处理，但这样不方便拓展到原问题，所以考虑直接对序列进行考察的判定方法。

从判定问题入手。先考虑如何判定一个序列是否是一个合法的欧拉序。当然可以模拟 dfs 的过程直接处理，但这样不方便拓展到原问题，所以考虑直接对序列进行考察的判定方法。

注意到任意满足 $a_{l_1} = a_{r_1}, a_{l_2} = a_{r_2}$ 的 $(l_1, r_1), (l_2, r_2)$ 都只能相离或包含，这是因为每个这样的区间都表示一个子树（或多个兄弟子树拼起来）。

除此之外，对于一对满足 $a_l = a_r$ 的 (l, r) ，必然有 $r - l = 2sz$ ，其中 sz 为该子树（或多个兄弟子树拼起来）的大小，从而区间 (l, r) 中必然有 sz 个不同元素（除了 $= a_l$ 的那些）。

似乎无法找到其它的必要条件，所以猜测这些条件是充分的，开始构造。如果已经出现不合法的 $(l_1, r_1), (l_2, r_2)$ ，或者有某一对 (l, r) 不满足 $2 \mid r - l$ ，则直接结束。否则我们先不管最后一个条件，而是在构造的过程中进行处理。

似乎无法找到其它的必要条件，所以猜测这些条件是充分的，开始构造。如果已经出现不合法的 $(l_1, r_1), (l_2, r_2)$ ，或者有某一对 (l, r) 不满足 $2 \mid r - l$ ，则直接结束。否则我们先不管最后一个条件，而是在构造的过程中进行处理。

找到一对 (l, r) ，使得 $a_l = a_r$ ，且 (l, r) 中已有元素互不相同。如果此时 (l, r) 中已有大于 sz 个不同元素，那么不合法。否则先使用没有用过的数填在任意空位直到填入了 sz 个不同的元素，如果这个过程中发现数不够用了，那么一定是某个其它区间不合法了，直接结束即可。

似乎无法找到其它的必要条件，所以猜测这些条件是充分的，开始构造。如果已经出现不合法的 $(l_1, r_1), (l_2, r_2)$ ，或者有某一对 (l, r) 不满足 $2 \mid r - l$ ，则直接结束。否则我们先不管最后一个条件，而是在构造的过程中进行处理。

找到一对 (l, r) ，使得 $a_l = a_r$ ，且 (l, r) 中已有元素互不相同。如果此时 (l, r) 中已有大于 sz 个不同元素，那么不合法。否则先使用没有用过的数填在任意空位直到填入了 sz 个不同的元素，如果这个过程中发现数不够用了，那么一定是某个其它区间不合法了，直接结束即可。

此时这个区间中有 sz 个不同元素已被填完，还剩下 $sz - 1$ 个空位需要填。注意到如果有相邻两个位置被填上，而它们的下一个位置是空的（即形如 $xy?$ ），则将 $?$ 变成 x ，那么这三个元素可以被缩成一个 x 。对于 $?xy$ 也是同理。这样不断缩起来最终得到的序列必然形如 $a?b?c?d$ 这样的形式，容易发现将 $?$ 全部换成 a_l 即可。

使用栈不断找到同一个方向上能合并的三元组，即可在 $O(sz)$ 时间内做完上述过程。此时对于原问题来说，从左往右扫描，在遇到 $a_l = a_r$ 时立即对 (l, r) 做上述过程（显然此时内部没有重复元素），然后这个区间就被缩成了一个点，继续向后即可。

最终可能还会剩下一个序列，这是因为整个树的根还没有被确定。对于这个情况，可以类似于已经确定根的情形来处理，唯一不同的地方在于最后不能直接填 a_l ，但此时缩完所有三元组之后要么第一个位置要么最后一个位置一定被确定了，所以将它作为根即可。

给定两棵 n 个点的树，构造一组点到点的映射 $i \rightarrow p_i$ ，使得若第一棵树上边 (i, j) ，那么第二棵树上没有边 (p_i, p_j) 。

数据范围： $n \leq 10^4$ 。

若某棵树是菊花那么显然无解。否则尝试构造。选取第一棵树的某个点作为根，然后从根开始向下逐个确定。

如果每次我们都将当前点映射到一个叶子上，那么这个点带来的限制至多只有一个点无法选择，这是非常小的一个限制。由于任意时刻还没有被选择的点形成一个连通块，所以除了最后一步以外都至少有两个限制。如果按照 bfs 序确定每个点，且每个点给后面的点带来的限制至多只有 1 个，因此除了最后一步以外我们总能找到合适的选择方式。

若某棵树是菊花那么显然无解。否则尝试构造。选取第一棵树的某个点作为根，然后从根开始向下逐个确定。

如果每次我们都将当前点映射到一个叶子上，那么这个点带来的限制至多只有一个点无法选择，这是非常小的一个限制。由于任意时刻还没有被选择的点形成一个连通块，所以除了最后一步以外都至少有两个限制。如果按照 bfs 序确定每个点，且每个点给后面的点带来的限制至多只有 1 个，因此除了最后一步以外我们总能找到合适的选择方式。

但最后一步可能唯一的一个能放的点受到了相邻点的约束。考虑对这个映射进行一些调整来使得方案合法。记 p_i 表示 i 映射到的点，记 q_i 表示映射到 i 的点。假如最后放在了第二棵树的节点 x 上，而现在唯一不合法的点对为 (x, y) ，则由于 x 是最后一步考虑的点， q_x 必然是叶子。

考虑将 q_x 改为某个 z ，只要满足两个条件： (y, p_z) 没有边，以及不存在一个 x 的邻居 u ，使得 (z, q_u) 有边。注意到叶子带来的限制会非常少。所以如果 p_z, z 分别为两棵树上的叶子，那么是比较方便的。

考虑将 q_x 改为某个 z ，只要满足两个条件： (y, p_z) 没有边，以及不存在一个 x 的邻居 u ，使得 (z, q_u) 有边。注意到叶子带来的限制会非常少。所以如果 p_z, z 分别为两棵树上的叶子，那么是比较方便的。

注意到现在我们还没有钦定根，于是在两棵树上各取一个叶子作为根，先将它们匹配，然后再做后面的过程。这样我们令 z 为第一棵树的根，限制就是非常少的。于是现在大部分情况直接令 q_x 为第一棵树的根 z 即可。

考虑将 q_x 改为某个 z ，只要满足两个条件： (y, p_z) 没有边，以及不存在一个 x 的邻居 u ，使得 (z, q_u) 有边。注意到叶子带来的限制会非常少。所以如果 p_z, z 分别为两棵树上的叶子，那么是比较方便的。

注意到现在我们还没有钦定根，于是在两棵树上各取一个叶子作为根，先将它们匹配，然后再做后面的过程。这样我们令 z 为第一棵树的根，限制就是非常少的。于是现在大部分情况直接令 q_x 为第一棵树的根 z 即可。

不能交换的情形有两种， (y, p_z) 有边，或存在 x 的邻居 u 使得 (z, q_u) 有边。对于第一种情形， x 的父亲是 y ， y 的父亲是根 p_z 。直接交换 p_z, y ，只需保证 y 的邻居中没有根的唯一一个儿子即可。如果钦定每次选择深度最大的叶子，那么根的儿子在第一步会被放在一个深度 ≥ 3 的点上（否则一定是菊花，不合法），从而此时是合法的。

对于第二种情形, q_u 一定是 z 在第一棵树上的唯一一个邻居。由于钦定了先放深度最大的叶子, 所以 u 必然是 x 的儿子而不可能是父亲。如果想要调整, 那么 u 不能乱动, 否则会带来更多的限制。因此考虑把 u 放到根上。这样把 x 换到 u 的位置, 然后把根放在 x 上即可。此时唯一不合法的情形就是 u 的邻居恰好占据了根唯一的儿子。

对于第二种情形, q_u 一定是 z 在第一棵树上的唯一一个邻居。由于钦定了先放深度最大的叶子, 所以 u 必然是 x 的儿子而不可能是父亲。如果想要调整, 那么 u 不能乱动, 否则会带来更多的限制。因此考虑把 u 放到根上。这样把 x 换到 u 的位置, 然后把根放在 x 上即可。此时唯一不合法的情形就是 u 的邻居恰好占据了根唯一的儿子。

这种情况可以把这个儿子也放进来一起交换。由于我们是按照深度从大往小放, 所以在放根的儿子是一定是只剩一条链了。又因为 q_u 自己就是第一棵树上根的儿子, 所以 u 是第一次被考虑的。那么要想 u 的儿子被放在根的儿子, 一定只能是这条链的周围全部都是 u 的儿子。这样在交换的时候就可以忽略很多限制了。

对于第二种情形, q_u 一定是 z 在第一棵树上的唯一一个邻居。由于钦定了先放深度最大的叶子, 所以 u 必然是 x 的儿子而不可能是父亲。如果想要调整, 那么 u 不能乱动, 否则会带来更多的限制。因此考虑把 u 放到根上。这样把 x 换到 u 的位置, 然后把根放在 x 上即可。此时唯一不合法的情形就是 u 的邻居恰好占据了根唯一的儿子。

这种情况可以把这个儿子也放进来一起交换。由于我们是按照深度从大往小放, 所以在放根的儿子是一定是只剩一条链了。又因为 q_u 自己就是第一棵树上根的儿子, 所以 u 是第一次被考虑的。那么要想 u 的儿子被放在根的儿子, 一定只能是这条链的周围全部都是 u 的儿子。这样在交换的时候就可以忽略很多限制了。

到这里对剩下的部分进行一些讨论以及手动构造即可, 细节不过多赘述。

给定两个 01 串 s, t ，一次操作可以选择 s 中的相邻两个位置 $|i - j| = 1$ ，将 s_i 添加至 s 的末尾，并删除 s_j 。要求通过不超过 $n + 1$ 次操作将 s 变为 t ，或输出无解。

数据范围： $|s| = |t| \leq 2 \times 10^5$ 。

首先忽略 $s = t$ 以及 s 不能操作的情况。倒过来考虑从 t 操作得到 s 。此时操作变为，选择一个和 t_n 相同的字符，将 t_n 删除并在那个位置的前面或后面插入任意一个 01 字符。如果 t 没有办法操作，则无解。

注意到 $n + 1$ 步操作足够让我们将整个串都删掉一遍并重新插入任意的字符，所以猜测一定有解。关键在于如何保证每次试图插入的时候在对应位置都能有和末尾相同的字符。

首先忽略 $s = t$ 以及 s 不能操作的情况。倒过来考虑从 t 操作得到 s 。此时操作变为，选择一个和 t_n 相同的字符，将 t_n 删除并在那个位置的前面或后面插入任意一个 01 字符。如果 t 没有办法操作，则无解。

注意到 $n + 1$ 步操作足够让我们将整个串都删掉一遍并重新插入任意的字符，所以猜测一定有解。关键在于如何保证每次试图插入的时候在对应位置都能有和末尾相同的字符。

由于我们希望在 t 的前面重新构造一个 s ，所以第一次操作应当是选择 t_1 并在它前面插入一个元素。但如果 $t_1 \neq t_n$ ，我们不能这么做。对于这种情况，如果 $t_1 = t_{n-1}$ ，就随便做一次操作，转化为 $t_1 = t_n$ 的情形。否则在 t_{n-1} 后面插入一个 t_1 ，同样可以转化为这种情形。

ARC207E Erase and Append 解析

现在还剩下 n 步操作，而此时 t 满足 $t_1 = t_n$ 。如果 $s_1 \neq s_n$ ，那么只需要根据 t_1, s_1 是否相同来决定下一次是插入在 t_1 的前面还是后面，一次操作后即可将 t 的前两个字符依次变为 s_1, s_n 。此时再依次考虑剩余的所有元素，一定可以选择当前两个元素之一进行操作，只需按照 s 的形态插入对应字符即可。这样做花费了 $n - 1$ 步，是可以接受的。

ARC207E Erase and Append 解析

现在还剩下 n 步操作，而此时 t 满足 $t_1 = t_n$ 。如果 $s_1 \neq s_n$ ，那么只需要根据 t_1, s_1 是否相同来决定下一次是插入在 t_1 的前面还是后面，一次操作后即可将 t 的前两个字符依次变为 s_1, s_n 。此时再依次考虑剩余的所有元素，一定可以选择当前两个元素之一进行操作，只需按照 s 的形态插入对应字符即可。这样做花费了 $n - 1$ 步，是可以接受的。

然而如果 $s_1 = s_n$ ，第一步不能直接填出 s_1 ，否则可能会导致下一步没法操作。注意到 s 一定满足存在至少两种不同字符（否则不合法），所以选择任意一个 $s_i \neq s_1$ ，如果我们通过两步将 t 的前三个字符变为 s_1, s_i, s_n ，则后续的问题转化为两个和 $s_1 \neq s_n$ 相同的子问题，同样可以解决。

ARC207E Erase and Append 解析

现在还剩下 n 步操作，而此时 t 满足 $t_1 = t_n$ 。如果 $s_1 \neq s_n$ ，那么只需要根据 t_1, s_1 是否相同来决定下一次是插入在 t_1 的前面还是后面，一次操作后即可将 t 的前两个字符依次变为 s_1, s_n 。此时再依次考虑剩余的所有元素，一定可以选择当前两个元素之一进行操作，只需按照 s 的形态插入对应字符即可。这样做花费了 $n - 1$ 步，是可以接受的。

然而如果 $s_1 = s_n$ ，第一步不能直接填出 s_1 ，否则可能会导致下一步没法操作。注意到 s 一定满足存在至少两种不同字符（否则不合法），所以选择任意一个 $s_i \neq s_1$ ，如果我们通过两步将 t 的前三个字符变为 s_1, s_i, s_n ，则后续的问题转化为两个和 $s_1 \neq s_n$ 相同的子问题，同样可以解决。

可以发现只有在 $t_1 \neq s_1$ ，且 $t_n \neq t_{n-1}$ 的情况下才不能直接通过两步完成构造。但上面的构造还有一次操作没有使用。可以在第一步时在最前面插入一个 s_1 ，转化为可以用两步完成构造的情形。这样就可以在总共 $n + 1$ 步之内完成构造。

有一张 n 个点 m 条边的无向图，其中一些点上存在棋子。定义一次操作为：对于每个棋子选择一个与它所在节点 u_i 相邻的点 v_i ，并同时将所有棋子移动到 v_i 。要求移动过后每个点上有至多一个棋子。给定初始局面 s 和最终局面 t ，问是否存在一个操作序列将 s 变成 t ，并给出长度至多为 $2n$ 的操作序列。

数据范围： $n \leq 2000, m \leq 10^4$ 。

先判掉一些平凡的情况，对于剩下的情形，仍然先从有解的充要条件入手。首先 s, t 这两个局面都必须能被操作至少一次（由于操作显然是可逆的，所以 t 也必须能被操作）。这个的判定可以通过二分图匹配来实现。即将每个点拆成两个点 i_0, i_1 ，然后对于每条边 (u, v) ，连边 (u_0, v_1) 和 (u_1, v_0) ，则需要判定这张图上是否存在 s 中的点作为左部点的满匹配以及 t 中的点作为左部点的满匹配。

先判掉一些平凡的情况，对于剩下的情形，仍然先从有解的充要条件入手。首先 s, t 这两个局面都必须能被操作至少一次（由于操作显然是可逆的，所以 t 也必须能被操作）。这个的判定可以通过二分图匹配来实现。即将每个点拆成两个点 i_0, i_1 ，然后对于每条边 (u, v) ，连边 (u_0, v_1) 和 (u_1, v_0) ，则需要判定这张图上是否存在 s 中的点作为左部点的满匹配以及 t 中的点作为左部点的满匹配。

其次对于每个连通块， s, t 在其中的棋子数目必须是一样的。同时由于每个棋子每次操作必须移动一步，所以如果某个连通块是二分图，那么所有原来在同一侧的棋子最后必须还是在同一侧。因此设该连通块中 s, t 在左侧和右侧的点数分别为 s_l, s_r 和 t_l, t_r ，那么必须有 $s_l = t_l, s_r = t_r$ 或 $s_l = t_r, s_r = t_l$ 。

对于每个这样的连通块，如果不满足 $s_l = t_l, s_r = t_r$ ，那么就意味着总操作步数必须是奇数。同理，若不满足第二个条件则意味着总操作步数必须是偶数。如果有两个连通块，一个要求是奇数，一个是偶数，那么也是不合法的。在这些条件下，取出第一步合法的操作，然后不断来回做这样的操作就可以一直进行下去，所以不会出现无路可走的情况。因此猜测在步数足够多的时候，总能一个一个将每个点归位，从而一定合法。

对于每个这样的连通块，如果不满足 $s_l = t_l, s_r = t_r$ ，那么就意味着总操作步数必须是奇数。同理，若不满足第二个条件则意味着总操作步数必须是偶数。如果有两个连通块，一个要求是奇数，一个是偶数，那么也是不合法的。在这些条件下，取出第一步合法的操作，然后不断来回做这样的操作就可以一直进行下去，所以不会出现无路可走的情况。因此猜测在步数足够多的时候，总能一个一个将每个点归位，从而一定合法。

下面考虑给出符合步数要求的构造。为了简化构造，我们尝试在保留上述条件的情况下简化这个图，即删掉一些没用的边，使得剩下的图仍然合法。先把原图换成我们求匹配时所建出的这个二分图。这样做的好处是我们的限制与二分图有关，这样直接将所有连通块都变成二分图，就不用分开讨论奇环的情况了，最后在输出答案时只需将路径中的 i_0, i_1 都看作点 i 即可。

此时有一个小问题就是 t 中的点在哪一侧。可以发现在原来判定是否有解时，我们可以确定操作步数的奇偶性（如果奇偶均可则不妨设为偶数即可），则根据这个奇偶性就可以得出 t 中的点应该在哪一侧。

此时有一个小问题就是 t 中的点在哪一侧。可以发现在原来判定是否有解时，我们可以确定操作步数的奇偶性（如果奇偶均可则不妨设为偶数即可），则根据这个奇偶性就可以得出 t 中的点应该在哪一侧。

注意到此时必然存在 s, t 的满匹配，所以我们取出求得的这两组匹配，然后保留其中的边。这里如果一条边在两次匹配中同时出现，将它保留两次，两次分别标记为在 s 和 t 中出现的。现在这个得到的图由若干环和链组成，且 s 中的边和 t 中的边交替出现。如果是一个环的话，那么 s, t 在这个环上的点一定都是间隔出现的，所以将环上所有 t 的匹配边换成所有 s 的匹配边，这仍然是一组合法的匹配。对每个环都做这个处理，则最后只剩下链以及二元环，将二元环看作重边，也转化为链的情形。

此时有一个小问题就是 t 中的点在哪一侧。可以发现在原来判定是否有解时，我们可以确定操作步数的奇偶性（如果奇偶均可则不妨设为偶数即可），则根据这个奇偶性就可以得出 t 中的点应该在哪一侧。

注意到此时必然存在 s, t 的满匹配，所以我们取出求得的这两组匹配，然后保留其中的边。这里如果一条边在两次匹配中同时出现，将它保留两次，两次分别标记为在 s 和 t 中出现的。现在这个得到的图由若干环和链组成，且 s 中的边和 t 中的边交替出现。如果是一个环的话，那么 s, t 在这个环上的点一定都是间隔出现的，所以将环上所有 t 的匹配边换成所有 s 的匹配边，这仍然是一组合法的匹配。对每个环都做这个处理，则最后只剩下链以及二元环，将二元环看作重边，也转化为链的情形。

唯一不对的事情就是图的连通性被破坏了。因此可以加入一些剩下的边形成一个极大生成森林。故原来的所有合法条件均被保留。因此只需要在这张图上找到一组解即可。

现在这张图是一棵树，且所有 s 中的点位于相同奇偶性的深度处，所有 t 中的点也是如此。考虑逐个将 s 中的点归位，则我们在考虑的过程中同时保留所有的匹配边，因为对于不用操作的点，它们应当在这条匹配边上来回走。

为了避免在移动的过程中出现干扰，我们考虑找到最靠外的一对匹配边，不妨设为 (t_u, t_v) ，同时找到距离它最近的 s 匹配边 (s_u, s_v) ，然后将 s_u 移动到 t_u 。这样一来，在 s 移动到 t 之后只需一直沿着匹配边来回走，由于 t 是最靠外的点，这样不会挡住其它点的路径。

移动的过程是容易的，但如果我们等上一对点移动到位了再启动下一个，操作步数是 $O(n^2)$ 的，所以必须将操作重叠起来进行。考虑为了保证不同路径之间不撞上所需要的最少的移动提前量是多少。由于选择的路径 $s \rightarrow t$ 是最外侧的 t 以及离它最近的 s ，对于任意一个其它的 s 来说，即使它向着这条路径的方向走，也至少需要与当前时刻相同的时间才能走到路径上。因此只需要我们让其它的 s 都晚出发至少一步，就一定不会发生碰撞。又因为有奇偶性的限制，晚两步即可。

移动的过程是容易的，但如果我们等上一对点移动到位了再启动下一个，操作步数是 $O(n^2)$ 的，所以必须将操作重叠起来进行。考虑为了保证不同路径之间不撞上所需要的最少的移动提前量是多少。由于选择的路径 $s \rightarrow t$ 是最外侧的 t 以及离它最近的 s ，对于任意一个其它的 s 来说，即使它向着这条路径的方向走，也至少需要与当前时刻相同的时间才能走到路径上。因此只需要我们让其它的 s 都晚出发至少一步，就一定不会发生碰撞。又因为有奇偶性的限制，晚两步即可。

考虑这样做的移动次数，有至多 $2n$ 个点和 n 对匹配，每一对匹配会多延时 2 步，故最大的总时间为 $2n$ （如果有在路径中需要走的步数，那么这些位置不存在匹配，所以相当于是从匹配数中扣除了一定的量加给了移动过程，总共仍然不超过 $2n$ ）。直接朴素实现，复杂度 $O(n^2 + \text{flow}(n, m))$ 。

交互题如何入手

需要明确自己需要什么样的信息。

- 根据操作次数限制，明确可以查询的规模
- 通过分治等方式利用有限的信息转化为子问题
- 从正面考虑，思考能找到什么样的信息
- 利用随机化等方式，尤其是交互库不自适应的时候

通信题如何入手

核心在于信息的压缩：

- 从理论上分析传递信息需要的 bit 数量
- 转化为计数问题后编码压缩
- 重复利用某些 bit，表示不同含义
- 可能损坏或乱序的信息：找到无损传递的方式

A 需要传送一个 $[0, 10^{18}]$ 的数给 B，他可以向 B 发送一个长度为 n 的 01 序列，但这个序列中有 k 个位置是坏的，B 收到的序列中这些位置的值始终为 0，其它位置则为 A 传送的值。A 知道哪些位置是坏的，但是 B 不知道。

数据范围： $n = 150, k \leq 40$ 。

直接传送这个数需要 60 位的信息，现在一共有 110 个空位。不过 B 不知道哪些位置是坏的，我们必须使用一些标记来指示用来传递信息的是哪些位。例如我们考虑将所有好的位分成若干连续段，每一段的开头放一个 1，则 B 看到第一个 1 就知道从这个位置开始，下面会有一段传送信息的区间。然而使用这种方式不能指示这一段到什么地方结束。

直接传送这个数需要 60 位的信息，现在一共有 110 个空位。不过 B 不知道哪些位置是坏的，我们必须使用一些标记来指示用来传递信息的是哪些位。例如我们考虑将所有好的位分成若干连续段，每一段的开头放一个 1，则 B 看到第一个 1 就知道从这个位置开始，下面会有一段传送信息的区间。然而使用这种方式不能指示这一段到什么地方结束。

为了解决这个问题，只能让两人提前约定好每一段的长度，这样 B 每看到一个 1，就知道从它开始向后的一段是用来传递信息的。这样的正确性是有保证的，但是不能充分利用坏位之间的位置，导致会造成很多浪费。

直接传送这个数需要 60 位的信息，现在一共有 110 个空位。不过 B 不知道哪些位置是坏的，我们必须使用一些标记来指示用来传递信息的是哪些位。例如我们考虑将所有好的位分成若干连续段，每一段的开头放一个 1，则 B 看到第一个 1 就知道从这个位置开始，下面会有一段传送信息的区间。然而使用这种方式不能指示这一段到什么地方结束。

为了解决这个问题，只能让两人提前约定好每一段的长度，这样 B 每看到一个 1，就知道从它开始向后的一段是用来传递信息的。这样的正确性是有保证的，但是不能充分利用坏位之间的位置，导致会造成很多浪费。

考虑进一步优化这个过程。首先注意到可以让两人将所有的位置按照同样的方式随机打乱，且可以将需要传输的数异或一个相同的随机数来将坏位置和待传输的数都变得完全均匀随机。这样可以规避给定的坏位置或者数有意对着做法卡的问题。

不妨规定每一段传输的长度为 2。如果某一个位是坏的，那么它其实也可以传输一位 0 的信息。所以在尝试向后匹配的时候，不一定需要找 3 个完整的空位。对于后两个信息位，如果某一位对应的信息是 0，那么我们允许这一位是坏的。由于传输的数也是随机的，加上这个优化之后效果会非常好。

不妨规定每一段传输的长度为 2。如果某一个位是坏的，那么它其实也可以传输一位 0 的信息。所以在尝试向后匹配的时候，不一定需要找 3 个完整的空位。对于后两个信息位，如果某一位对应的信息是 0，那么我们允许这一位是坏的。由于传输的数也是随机的，加上这个优化之后效果会非常好。

进一步地，如果我们可以多次随机，取最优的一次，那么这个正确率是足够的。问题在于 A 多次随机的时候到 B 那里就不知道是采用了哪种随机方式了，故无法进行还原。

不妨规定每一段传输的长度为 2。如果某一个位是坏的，那么它其实也可以传输一位 0 的信息。所以在尝试向后匹配的时候，不一定需要找 3 个完整的空位。对于后两个信息位，如果某一位对应的信息是 0，那么我们允许这一位是坏的。由于传输的数也是随机的，加上这个优化之后效果会非常好。

进一步地，如果我们可以多次随机，取最优的一次，那么这个正确率是足够的。问题在于 A 多次随机的时候到 B 那里就不知道是采用了哪种随机方式了，故无法进行还原。

在第 i 次随机的时候，要求填完所有信息，后面仍有 $i-1$ 个空位，则我们将这 $i-1$ 个空位全部填上 1。这样 B 读完所有信息位之后，只需要数一下后面有多少个 1，就知道是采用的哪种随机方式了。注意这种方式中只能更改对查询数的随机方式，而不能改变序列的随机方式。经过测试，可以通过。

有一个未知的 n 阶排列 p , 满足 $p_i \neq i$, 需要在 $10n$ 次询问内求出它。
在所有询问前, 需要给出一个常数 $1 \leq k \leq n$, 然后每次询问给出一个排列 q_i , 交互库会回答满足 $i < j, p_{q_i} = q_j, i \neq k$ 的二元组 (i, j) 数目。

数据范围: 交互库非自适应, $4 \leq n \leq 100$ 。

考虑每次钦定一个特殊的元素 i 放在 k 这个位置，通过返回的值和不包含 k 的答案来确定 p_i 是否等于某个 $j > k$ 的 q_j ，从而形成一个类似二分的过程。

考虑每次钦定一个特殊的元素 i 放在 k 这个位置，通过返回的值和不包含 k 的答案来确定 p_i 是否等于某个 $j > k$ 的 q_j ，从而形成一个类似二分的过程。

但我们并不知道不包含 k 的答案应该是什么，所以只能通过几次询问的答案加减来抵消掉 i 以外的元素的贡献。这样我们可以找到一个排列 q 满足 $q_k = i$ ，先问一次 q ，然后把除了 q_k 以外的部分翻转，再问一次。令 j 为 p_i^{-1} 在 q 中的位置，可以推断出 $[j < k] + [n - j < k]$ 的值。

考虑每次钦定一个特殊的元素 i 放在 k 这个位置，通过返回的值和不包含 k 的答案来确定 p_i 是否等于某个 $j > k$ 的 q_j ，从而形成一个类似二分的过程。

但我们并不知道不包含 k 的答案应该是什么，所以只能通过几次询问的答案加减来抵消掉 i 以外的元素的贡献。这样我们可以找到一个排列 q 满足 $q_k = i$ ，先问一次 q ，然后把除了 q_k 以外的部分翻转，再问一次。令 j 为 p_i^{-1} 在 q 中的位置，可以推断出 $[j < k] + [n - j < k]$ 的值。

不妨设 $k < \frac{n}{2}$ ，则这两个条件不可能同时满足，因此相当于可以确定 p_i 是否位于 $(k, n - k)$ 这个区间内，取 $k = \frac{n}{4}$ ，即可每次缩小一半 p_i 的范围。这样可以做到 $2n \log n$ 次询问。

两次询问其实可以提供一个值为 $0/1/2$ 的信息，但在上述构造中，我们只允许答案出现 $0/1$ ，而舍弃了 2 的可能性，这是不优的。我们尝试在询问 q 和 $\text{rev}(q)$ 的时候把不同的元素放在第 k 个位置，这样它们的 p_i 对应位置所属区域是独立的，故 $0/1/2$ 都是可以出现的。

两次询问其实可以提供一个值为 $0/1/2$ 的信息，但在上述构造中，我们只允许答案出现 $0/1$ ，而舍弃了 2 的可能性，这是不优的。我们尝试在询问 q 和 $\text{rev}(q)$ 的时候把不同的元素放在第 k 个位置，这样它们的 p_i 对应位置所属区域是独立的，故 $0/1/2$ 都是可以出现的。

令 $k = \frac{n}{2} + 1$ ，然后选两个元素 x, y ，钦定 $q_k = x, q_{k-1} = y$ 。假设 $q_{i > k+1}$ 的部分形成集合 A ， $q_{i < k}$ 的部分形成集合 B ，则同时询问 q 和 $\text{rev}(q)$ ，我们可以得知 $[p_y \in A] + [p_x \in B]$ 的值。

如果这个值是 $0/2$ ，那我们就同时将 p_x, p_y 的取值范围减半了。每次随机将所有数分成 A, B 两部分，则答案为 1 的概率是 $1/2$ ，所以期望 2 次就能继续分治下去。

如果 p_x, p_y 的取值范围之和加起来 $< n - 2$ 且不交，将它们的取值分别随机分成均等的两部分（设为 X_1, X_2, Y_1, Y_2 ），然后先令 $A = X_1, B = Y_1$ 进行一次询问，如果结果不为 0/2，那我们可以断言 $A = X_1, B = Y_2$ 时结果一定为 0/2。所以这样期望的步数变成了 1.5，总步数即为 $1.5n \log n$ 。实现时有一些细节，不再赘述。

有一个未知的 n 阶排列 p_i , 记 $q = p^{-1}$ 满足 $q_{p_i} = i$, 需要求出这个排列。可以进行四种询问, 其中每次询问给出若干个互不相同的下标 $i_1, \dots, i_m$, 交互库会求出集合 S , 其中第 1, 3 种询问中 $S = \{p_{i_j}\}$, 第 2, 4 种询问中 $S = \{q_{i_j}\}$ 。然后有参数 k , 对于第 1, 2 种询问, $k = \min(60, m)$, 对于第 3, 4 种询问, $k = \min(300, m)$ 。交互库会返回 S 中最大的 k 个数的值。需要在总共不超过 30 次询问, 且第 3, 4 种询问不超过一次的情况下求出排列。

数据范围: $n \leq 890$, 交互库非自适应。

先进行理论分析，如果只使用 1, 3 操作，则一共可以知道 $29 \times 60 + 300 = 2040$ 个形如 “ $p_i = x$ 在某次询问中出现了”，然而如果想要通过这些信息唯一推断出每个集合是什么，相当于需要将每个 i 放在不同的询问集合中，但 $n = 890$ 时至少也需要总共 2172 个询问，因此没有理论推断出答案的可能性。

先进行理论分析，如果只使用 1, 3 操作，则一共可以知道 $29 \times 60 + 300 = 2040$ 个形如 “ $p_i = x$ 在某次询问中出现了”，然而如果想要通过这些信息唯一推断出每个集合是什么，相当于需要将每个 i 放在不同的询问集合中，但 $n = 890$ 时至少也需要总共 2172 个询问，因此没有理论推断出答案的可能性。

所以必然是需要同时利用 1, 2, 3, 4 这些操作。先假定我们能够获得整个答案集合，思考一下同时询问 1, 2 意味着什么。假如我们先问了一些 1，那么我们去问 2 的时候一定要利用这些 1 操作的结果，否则如果按照自己的方式提问，这些结果无法与 1 操作获知的信息结合。

先进行理论分析，如果只使用 1, 3 操作，则一共可以知道 $29 \times 60 + 300 = 2040$ 个形如 “ $p_i = x$ 在某次询问中出现了”，然而如果想要通过这些信息唯一推断出每个集合是什么，相当于需要将每个 i 放在不同的询问集合中，但 $n = 890$ 时至少也需要总共 2172 个询问，因此没有理论推断出答案的可能性。

所以必然是需要同时利用 1, 2, 3, 4 这些操作。先假定我们能够获得整个答案集合，思考一下同时询问 1, 2 意味着什么。假如我们先问了一些 1，那么我们去问 2 的时候一定要利用这些 1 操作的结果，否则如果按照自己的方式提问，这些结果无法与 1 操作获知的信息结合。

注意到如果若干位置 $i_1, i_2, \dots, i_k$ 被完全相同的 1 操作集合询问了，那么我们可以确定 $\{p_{i_1}, \dots, p_{i_k}\}$ 这个集合，但是不知道具体如何对应的。此时我们反过来询问 p_{i_1} ，就可以知道它对应的 i_1 是什么，以此类推。

利用 $m > k$ 这种操作，我们可以确定不止 k 个位置的值。假设所有未确定的等价类大小都为 2，即有若干对 (i_1, i_2) ，已知它们对应 (p_1, p_2) ，但不知怎么对应。注意到 (i_1, i_2) 的分组可以按照我们的想法自由进行，但是 (p_1, p_2) 不能由我们决定。所以我们试图将 i_1, i_2 按照一些好的方式分组，然后一下询问所有组的 p_1 对应的位置是什么。

利用 $m > k$ 这种操作，我们可以确定不止 k 个位置的值。假设所有未确定的等价类大小都为 2，即有若干对 (i_1, i_2) ，已知它们对应 (p_1, p_2) ，但不知怎么对应。注意到 (i_1, i_2) 的分组可以按照我们的想法自由进行，但是 (p_1, p_2) 不能由我们决定。所以我们试图将 i_1, i_2 按照一些好的方式分组，然后一下询问所有组的 p_1 对应的位置是什么。

注意到一次回答能给予我们的信息除了“ x 在集合中”以外，还有“ x 不在集合中”这样的信息。如果某一组 (i_1, i_2) 虽然都没有出现在集合中，但是其中较大的一个（不妨设为 i_2 ）比答案集合的最小值要大，那我们就知道 i_2 一定不和 p_1 匹配，否则它一定会出现在答案集合中。所以我们可以考虑将大的下标和小的下标组合在一起。

CF2129F2 Top-K Tracker 解析

具体来说，设一共有 m 对 (i_1, i_2) ，那将 $(i, 2m - i + 1)$ 进行配对。这样只要答案集合的第 k 大下标不大于 m ，就能够唯一确定每一对中 $> m$ 的那个下标是否和对应的 p_1 匹配。因此随机打乱每一对，这样相当于每一次询问只有 $\frac{1}{2}$ 的概率产生贡献，期望的贡献是 $\frac{m}{2}$ ，即只要 $\frac{m}{2}$ 比 k 小足够多，就能够以极大概率求出答案。

具体来说，设一共有 m 对 (i_1, i_2) ，那将 $(i, 2m - i + 1)$ 进行配对。这样只要答案集合的第 k 大下标不大于 m ，就能够唯一确定每一对中 $> m$ 的那个下标是否和对应的 p_1 匹配。因此随机打乱每一对，这样相当于每一次询问只有 $\frac{1}{2}$ 的概率产生贡献，期望的贡献是 $\frac{m}{2}$ ，即只要 $\frac{m}{2}$ 比 k 小足够多，就能够以极大概率求出答案。

因此使用那一次 $k = 300$ 的操作作为最后一次操作，在前面利用 $k = 60$ 的操作做好分组。可以发现只使用大小为 $0, 1, 2$ 的集合能够凑出 436 种不同的询问集合，即可以得到 436 组不同的元素。而 $m = 436$ 满足 $\frac{m}{2} < k$ ，所以后面的询问是可以进行的。

然而这样只确定了 $436 \times 2 = 872$ 种元素，不够 890 个。但可以发现现在每次询问都还剩下 3 个位置没有使用，所以利用这些位置凑出 29 个不同的三元集合，可以直接确定答案。这样恰好确定了 891 种元素，符合要求。

给定一个长度为 n 的排列 a_i , 对于一个 01 序列 s , 定义 t 满足 $t_i = s_{a_i}$ 。现在已知 a , 需要进行若干轮操作, 将 s 变成 t 。奇数轮操作中, 会从 1 开始依次遍历 s 的每个位置, 每次会得知 s_i 的值, 并需立即决定将其赋值为 0/1。在做出决定时只知道之前的所有 s_i , 而不知道后面的值。偶数轮操作中会倒着做上述过程。要求在 3 轮操作内将 s 变为 t 。

数据范围: $n \leq 500$ 。

本质还是传递信息：通过前面的轮次给后面合适的信息，使得最后一轮能直接复原。如果在第一轮操作的时候直接赋值，赋第一个值的时候就会导致被赋值位置的信息丢失，所以只能考虑利用异或表示信息。

本质还是传递信息：通过前面的轮次给后面合适的信息，使得最后一轮能直接复原。如果在第一轮操作的时候直接赋值，赋第一个值的时候就会导致被赋值位置的信息丢失，所以只能考虑利用异或表示信息。

第一轮操作中我们可以将一个位置的值修改为若干它以及它之前位置的值的异或。由于需要能唯一还原解需要所有位置表示的值线性无关，每个位置的组成都必须包含其本身。同理，为了能在第三轮操作的时候从左到右依次还原出解，第二轮操作之后 i 位置一定由若干 $j \leq i$ 的 a_j 异或而来，且必须包含 a_i 。

本质还是传递信息：通过前面的轮次给后面合适的信息，使得最后一轮能直接复原。如果在第一轮操作的时候直接赋值，赋第一个值的时候就会导致被赋值位置的信息丢失，所以只能考虑利用异或表示信息。

第一轮操作中我们可以将一个位置的值修改为若干它以及它之前位置的值的异或。由于需要能唯一还原解需要所有位置表示的值线性无关，每个位置的组成都必须包含其本身。同理，为了能在第三轮操作的时候从左到右依次还原出解，第二轮操作之后 i 位置一定由若干 $j \leq i$ 的 a_j 异或而来，且必须包含 a_i 。

考虑一些比较简单的构造方式，例如令 i 位置的值由至多两个元素异或得到。这么做是因为最终我们需要把 i 变成 a_i ，而如果进行连边 $i \rightarrow a_i$ ，可以将任意两个元素的异或拆分为若干对 (i, a_i) 的异或合起来的结果，故这样能够建立起 i 与 a_i 的联系。

下面需要对于一个 i , 找到一个 $j < i$, 使得第一轮操作后 i 由原来的 i, j 组成, 再找到一个 $k < i$, 使得第二轮操作后 i 由原来的 a_i, a_k 组成。考虑找一个中间状态 (i, a_i) , 即在第二轮从 (i, j) 转化为 (a_i, a_k) 的过程中, 借助中间状态 (i, a_i) , 先从 (i, j) 转化到 (i, a_i) , 再从 (i, a_i) 转化到 (a_i, a_k) 。

下面需要对于一个 i , 找到一个 $j < i$, 使得第一轮操作后 i 由原来的 i, j 组成, 再找到一个 $k < i$, 使得第二轮操作后 i 由原来的 a_i, a_k 组成。考虑找一个中间状态 (i, a_i) , 即在第二轮从 (i, j) 转化为 (a_i, a_k) 的过程中, 借助中间状态 (i, a_i) , 先从 (i, j) 转化到 (i, a_i) , 再从 (i, a_i) 转化到 (a_i, a_k) 。

如果 $a_i < i$, 那么可以直接将 i 设定为 (i, a_i) 。

否则如果 $a_{a_i} < i$, 可以将 i 设为 (i, a_{a_i}) , 并将 a_i 也设定为 (a_i, a_{a_i}) 。注意到对于 a_i 来说, (a_i, a_{a_i}) 相当于是有一个 (j, a_j) 这样的组合, 所以对于它也是合理的。以此类推, 令 nxt_i 为不断跳 $i \rightarrow a_i$ 所到达的第一个 $< i$ 的点, 则将 i 设为 (i, nxt_i) 一定合法。特别地, 如果 nxt_i 不存在, 则认为 i 仅由它自己组成。

考虑这样构造之后如何在第二轮先将 (i, nxt_i) 变成 (i, a_i) 。

首先如果 $a_i < i$, 必然有 $\text{nxt}_i = a_i$, 故已经合法了。另一方面, 我们可以从 $j = a_i$, 不断跳 nxt_j 并将 (j, nxt_j) 异或到 i 的答案中, 直到 $j = \text{nxt}_i$ 。显然一定会出现这样的时刻, 因为在 $i \rightarrow \text{nxt}_i$ 中间不存在任何 $< i$ 的点, 故为了超越 nxt_i 必须要经过 nxt_i 本身。这样就构造出了 (i, a_i) 。

另一侧也是类似的, 令 pre_i 表示反着跳第一个 $< i$ 的点, 则可以通过和上面一样的方式从 (i, a_i) 变成 (a_{pre_i}, a_i) 。这样就完成了构造。对于每个环最小的点, 由于其原来只有它本身, 需要特殊处理一下, 不过这部分是容易的。

有 n 张写有 01 的卡片，A 会依次一张张抽出。每次抽出一张，A 需要选择是丢弃它，还是将它插入到答案序列的任意位置。操作完毕后，B 会看到答案序列，他需要正确报告原始的 n 张卡片中有多少张 1。

数据范围： $n = 900$ ，要求答案序列的长度不超过 14。

QOJ11402 Fortune Telling 3 解析

先尝试得到一个 $O(\log n)$ 的解法。主要的问题在于，一方面我们无法在过程中得知最后总共有多少个 1，另一方面我们后面得到的卡片也不一定有足够的 0/1 来表示出答案的二进制。

注意到越往后我们知道的信息越完全，所以尽可能在后面放置有效的信息。考虑一个界 B ，在前 $n - B$ 张牌处，我们进行观察，然后把后 B 张牌全部放在序列中，通过某种方式表示出前 $n - B$ 张牌中 1 个数的信息。此时由于后 B 张牌全部在序列中，所以 B 只需要数一下这里面 1 的个数，再加上这些牌表示的信息，就可以知道答案。但还需要解决牌数量不够的问题。

QOJ11402 Fortune Telling 3 解析

先尝试得到一个 $O(\log n)$ 的解法。主要的问题在于，一方面我们无法在过程中得知最后总共有多少个 1，另一方面我们后面得到的卡片也不一定有足够的 0/1 来表示出答案的二进制。

注意到越往后我们知道的信息越完全，所以尽可能在后面放置有效的信息。考虑一个界 B ，在前 $n - B$ 张牌处，我们进行观察，然后把后 B 张牌全部放在序列中，通过某种方式表示出前 $n - B$ 张牌中 1 个数的信息。此时由于后 B 张牌全部在序列中，所以 B 只需要数一下这里面 1 的个数，再加上这些牌表示的信息，就可以知道答案。但还需要解决牌数量不够的问题。

必须构造一种方案使得无论下一张牌是什么，都能把它放进牌堆的某个位置，表示出我们想要的信息。考虑将二进制表示看作一个形如 '0|1|1|0|0' 的形式，即中间有分隔符。如果我们已有固定的分隔符，就可以令 0/1 字符不再表示一个值，而是表示一个计数器，可以计数对应位的值是 0 还是 1。

下面我们只需要利用已有的前 $n - B$ 个字符搞出分隔符即可。注意到我们可以提前构造出 101010 这样的串，其中每一对 10 表示一个分隔符。容易看出无论我们在两对 10 之前插入 0 还是 1，总有唯一还原的办法。因此取 $B = \log n$ ，即可做到 $3B = 30$ 。

下面我们只需要利用已有的前 $n - B$ 个字符搞出分隔符即可。注意到我们可以提前构造出 101010 这样的串，其中每一对 10 表示一个分隔符。容易看出无论我们在两对 10 之前插入 0 还是 1，总有唯一还原的办法。因此取 $B = \log n$ ，即可做到 $3B = 30$ 。

二进制表示是很浪费的，因为 0/1 已经变成计数器了，那么没必要表示每一位只有 0/1 的东西。考虑进行信息压缩：如果使用 B_0 个分隔符，最后留下 B_1 个 0/1 字符插入，可以表示出 $\binom{B_0+B_1}{B_1}$ 种不同的信息。当 $B_0 = 4, B_1 = 10$ 时可以表示的信息数目 > 900 ，且 $2B_0 + B_1$ 最小，这样就得到了一组 18 的构造。进一步压缩利用 $< B_1$ 的长度，可以做到 16。

考虑从最后的 B_1 个字符开始，动态决定什么时候转变至下一个阶段。如果现在还剩下 B_1 个字符，且我们已经知道倒数第 B_1 个字符是什么。如果前面的 1 的总数对应的编码是应当使用 B_1 位来表示的，就立即转入下一阶段，把最后的 B_1 个字符插入对应的位置。

否则保留这个字符，并将它视为前面一段的一部分，即它会贡献到最后需要表示的数中。接着考虑倒数第 $B_1 - 1$ 个位置，并根据当前需要表示的信息对应编码是否有 $B_1 - 1$ 来决定是否进入下一阶段，以此类推。

考虑从最后的 B_1 个字符开始，动态决定什么时候转变至下一个阶段。如果现在还剩下 B_1 个字符，且我们已经知道倒数第 B_1 个字符是什么。如果前面的 1 的总数对应的编码是应当使用 B_1 位来表示的，就立即转入下一阶段，把最后的 B_1 个字符插入对应的位置。

否则保留这个字符，并将它视为前面一段的一部分，即它会贡献到最后需要表示的数中。接着考虑倒数第 $B_1 - 1$ 个位置，并根据当前需要表示的信息对应编码是否有 $B_1 - 1$ 来决定是否进入下一阶段，以此类推。

可以发现，只要我们让越大的数用越少的位数来表示，就可以没有重叠地用所有编码的可能性覆盖所有数。即可以给每一种位数赋一个代表的区间 $[L_i, R_i]$ ，使得 $L_{i-1} = R_i + 1$ ，且 $[L_i, R_i]$ 的每个数分别对应 $\binom{B_0+i}{B_0}$ 个不同编码中的一个。

QOJ11402 Fortune Telling 3 解析

借鉴这个想法，我们尝试将 B_0 也变动起来。这里最大的问题在于分隔符是需要预先放置的，否则可能会出现 0/1 不够的问题。但我们在一开始不知道究竟有多少个 1，也自然不知道需要放多少个分隔符。

因此只能在 1 的个数不断增加的过程中，每当 1 的个数超过了某个阈值，就多放一个分隔符。不过一个分隔符由一对 $(0, 1)$ 组成，但如果这之后所有的字符都是 1，就没法放分隔符了。不过当 1 的个数刚好超过阈值的时候，当前这个字符一定是 1，所以只有可能 0 的个数不够。因此我们无论如何都先放 B_0 个 0，然后通过动态调整 1 的个数来调整分隔符的个数。

QOJ11402 Fortune Telling 3 解析

借鉴这个想法，我们尝试将 B_0 也变动起来。这里最大的问题在于分隔符是需要预先放置的，否则可能会出现 0/1 不够的问题。但我们在一开始不知道究竟有多少个 1，也自然不知道需要放多少个分隔符。

因此只能在 1 的个数不断增加的过程中，每当 1 的个数超过了某个阈值，就多放一个分隔符。不过一个分隔符由一对 $(0, 1)$ 组成，但如果这之后所有的字符都是 1，就没法放分隔符了。不过当 1 的个数刚好超过阈值的时候，当前这个字符一定是 1，所以只有可能 0 的个数不够。因此我们无论如何都先放 B_0 个 0，然后通过动态调整 1 的个数来调整分隔符的个数。

这样假定已经设计好了阈值，一直观察到最后的 B_1 张牌为止。为了避免问题，我们认为这之后所有的时刻都不会再添加分隔符，而是直接转入考察需要如何填入后面字符的阶段。经过计算，这样压缩时合理设置 B_0, B_1 的界，可以在 14 位内表示出全部信息，细节较多，可以参考原题题解。

有一个长度为 n 的 01 序列 a , 已知 $a_0 = 1$ 。可以进行 Q 次询问, 每次询问给定若干互不相同的下标 $i_1, i_2, \dots, i_k$, 可以得知 $\sum [a_{i_j} \neq a_{i_{j+1}}]$ 的值。要求算出 $\sum a_i$ 的值是多少。

数据范围: $n \leq 2 \times 10^4, Q = 226$ 。

考虑一次询问能获得什么信息。注意到如果有连续两个未知的位置被我们放入了同一组询问中（假设对应子序列为 x, A, B ，其中 x 是任意元素， A, B 都是未知的），那么在 $x \neq B$ 的时候， A 的两种取值对答案的贡献完全相同。所以这种情况是没用的。

因此我们询问的形式只能是 $x, A, y, B, \dots$ ，其中 x, y 为已知的位置， A, B 为未知的位置。可以发现如果 $x \neq y$ ，同样会出现上面的问题，因此必须要求 $x = y$ 。此时 A 的两种取值对答案的贡献分别为 $0/2$ ，因此一次形如 $0, A, 0, B, 0, C, \dots$ 的询问可以确定 $A, B, C, \dots$ 中总共有多少个 1。

考虑一次询问能获得什么信息。注意到如果有连续两个未知的位置被我们放入了同一组询问中（假设对应子序列为 x, A, B ，其中 x 是任意元素， A, B 都是未知的），那么在 $x \neq B$ 的时候， A 的两种取值对答案的贡献完全相同。所以这种情况是没用的。

因此我们询问的形式只能是 $x, A, y, B, \dots$ ，其中 x, y 为已知的位置， A, B 为未知的位置。可以发现如果 $x \neq y$ ，同样会出现上面的问题，因此必须要求 $x = y$ 。此时 A 的两种取值对答案的贡献分别为 $0/2$ ，因此一次形如 $0, A, 0, B, 0, C, \dots$ 的询问可以确定 $A, B, C, \dots$ 中总共有多少个 1。

确定一个阈值 B ，先进行最简单的询问（即 $\{0, x\}$ ）来得到至少 B 个相同的元素。然后把剩下的元素每 B 个分为一组，每组用已有的那些相同元素进行询问，则可以确定它们的和。由于为了确保 B 个相同元素需要至多 $2B - 1$ 次询问，所以总次数约为 $2B - 1 + \frac{n}{B}$ ，即 $2\sqrt{2n}$ 。

QOJ1138 Counting Mushrooms 解析

下面可以发现，我们可以通过 $\{0, x, 0, y\}$ 这样的询问同时确定 x, y 。这是因为这样的询问得到的结果是 $2x + y$ 的值，由于 x, y 都是 $0/1$ ，故可以唯一确定。这样前面的询问次数就变为 $B - 1$ ，总次数变为 $2\sqrt{n}$ 。

下面可以发现，我们可以通过 $\{0, x, 0, y\}$ 这样的询问同时确定 x, y 。这是因为这样的询问得到的结果是 $2x + y$ 的值，由于 x, y 都是 0/1，故可以唯一确定。这样前面的询问次数就变为 $B - 1$ ，总次数变为 $2\sqrt{n}$ 。

然后类似于上面的优化，如果我们在每次询问的最后都加一个未知元素，那么它对答案的贡献为 0/1，其它元素对答案的贡献为 0/2，故通过奇偶性可以确定最后一个元素的值，以及所有元素的和。这样我们每次询问时能比上一次多一个已知元素。

但由于已知元素可能不一定是当前使用的这种数，所以每两次才会真的增加一个元素供询问。这样后面的次数即为

$B + B + (B + 1) + (B + 1) + \cdots \geq n$ 所需的最少项数。总步数约为 $1.75\sqrt{n}$ ，经实际测试为 244。

再回顾我们的询问，发现一次询问中只用了一种已知元素，但我们对于询问的要求仅仅是不同元素之间不能插入未知元素。但不同的元素是可以放在相邻位置来拼出两个询问的。例如考虑进行询问

$0, x, 0, y, 0, 1, z, 1, u$ ，则这样的询问能得知 u 的值以及 $x + y + 1 - z$ 的值。

注意到我们只要再知道 $y + z$ 的值，就能知道 $x + 2y$ 的值。而 x, y 都是 $0/1$ ，所以能唯一确定 x, y, z ，而不需要通过三个方程来解。这样是一种通过 2 步问出 4 个元素的方法。但在另一次询问中，只需要问出 $y + z$ 的值，所以可以再带一个元素，即用 $0, y, 0, z, 0, v$ 这样的询问。

可以发现这样不仅能确定 $y + z$ 的值，也能确定 v 的值。故这样通过两次询问问出了 5 个数，前面的步数被减少了 $1/5$ ，取最优的阈值，恰好是 226 步。

谢谢大家

谢谢大家！